Web Development Practice — Project-Level Problems

These problems are designed for college students who have completed the corresponding part of their coding journey and want to build real, confidence-boosting projects on their own. Each project simulates a real-world use case and should be implemented as a complete, working application — not just a code snippet.

Six tracks are covered below, progressing in the same order most students learn web development: HTML+CSS, HTML+CSS+JS, React+Node, React+Node+AI, React+Node+MongoDB, and React+Node+MongoDB+AI. Each track has 5 projects — 2 Easy, 2 Medium, 1 Hard — so you can pick a project that matches exactly where you are right now.

How to Use This Document

Pick **one project per track** to start. Do not try to build everything at once. Finish a project completely — including styling, edge cases, and a working demo — before moving to the next one. A fully finished Easy project teaches more than a half-finished Hard project.

If a project description mentions login, users, or saved data but the track does not include a backend or database, simulate that behavior using in-browser storage or hardcoded arrays — the goal is to practice the structure, not to fake having a backend you don't have yet.

Track 1: HTML + CSS

Pure structure and styling. No interactivity, no JavaScript. The goal is to practice semantic HTML, layout (Flexbox/Grid), responsive design, and visual polish.

1.1 — Restaurant Menu Card (Easy)

Description: A single-page visual menu for a restaurant like "Sharma Family Dhaba" — showing food categories (Starters, Main Course, Desserts, Beverages) with item name, price, and a short description for each dish.

Sections to build:

- Header with restaurant name and tagline
- Category tabs or section headers (Starters, Main Course, Desserts, Beverages)
- Item cards showing dish name, price in ₹, and description
- Footer with address and contact details

Focus skills: Semantic HTML tags, CSS Grid or Flexbox for the card layout, consistent spacing and typography.

1.2 — Personal Portfolio Landing Page (Easy)

Description: A one-page portfolio site for a fictional student, e.g. "Suresh Kumar — B.Tech CSE, Final Year." Showcases skills, projects, and contact information.

Sections to build:

- Hero section with name, role, and a short intro
- Skills section showing technologies as badges or tags
- Projects section with 3 project cards (title, description, tech used)
- Contact section with email, LinkedIn, GitHub links

Focus skills: Page structure, responsive design with media queries, hover effects using pure CSS.

1.3 — Indian Railways Ticket Booking Display (Medium)

Description: A static visual layout that mimics an IRCTC-style train ticket search results page — showing multiple train options with departure/arrival times, duration, and seat availability status.

Sections to build:

- Search bar layout (From station, To station, Date — visual only, no functionality)
- A list of 4–5 train result cards, each showing train name, number, departure/arrival time, duration, and class-wise availability (Sleeper, AC 3-Tier, AC 2-Tier)
- Status badges (Available, Waiting List, RAC) with different colors
- Responsive layout that stacks cleanly on mobile

Focus skills: Complex multi-column layouts with CSS Grid, conditional-looking styling using classes, responsive breakpoints.

1.4 — E-Commerce Product Listing Page (Medium)

Description: A product grid page similar to a Flipkart or Amazon category page, showing a list of products (e.g. "Mobile Phones") with images, names, prices, ratings, and a "View Details" button.

Sections to build:

- Top navigation bar with logo, search bar (visual), and cart icon
- Filter sidebar (Brand, Price Range, Rating — visual only)
- Product grid with at least 8 product cards
- Pagination control at the bottom (visual only)

Focus skills: Sidebar + main content layout, CSS Grid for the product cards, consistent card design system, image aspect ratio handling.

1.5 — Multi-Page College Website (Hard)

Description: A complete multi-page static website for a fictional institute, e.g. "Brain Mentors Institute of Technology," with separate HTML pages linked together: Home, About, Courses, Faculty, and Contact.

Pages to build:

- Home — hero banner, highlights, call-to-action
- About — institute history, mission, vision
- Courses — grid of course cards (e.g. MERN Stack, Data Analytics, Cloud Computing) with duration and fee details
- Faculty — grid of faculty member cards with photo placeholder, name, designation
- Contact — address, map placeholder, contact form layout (visual only)

Focus skills: Consistent shared header/footer across multiple files, a unified design system (colors, spacing, fonts) reused across pages, real-world site information architecture, responsive design at scale.

Track 2: HTML + CSS + JavaScript

Same visual builds as Track 1, now made interactive. The goal is DOM manipulation, event handling, and basic client-side logic — still no backend.

2.1 — Interactive To-Do List (Easy)

Description: A task manager where users can add, complete, and delete tasks. Data does not need to persist after refresh unless you choose to add `localStorage` as a bonus.

Features:

- Input box and "Add Task" button
- List of tasks with a checkbox to mark complete (strikethrough style)
- Delete button per task

- Task counter showing "X tasks remaining"

Focus skills: DOM manipulation, event listeners, array operations (push, filter, map) to manage task state in JavaScript.

2.2 — BMI Calculator (Easy)

Description: A simple health calculator where a user enters height (cm) and weight (kg) and instantly sees their BMI value along with a category label (Underweight, Normal, Overweight, Obese).

Features:

- Two input fields: height and weight
- Calculate button
- Result display showing the BMI number and category, with category-based color (e.g. green for Normal, orange for Overweight)
- Input validation — reject empty or negative values with a clear message

Focus skills: Form handling, basic math logic in JavaScript, conditional styling based on calculated values.

2.3 — Quiz Application (Medium)

Description: A multiple-choice quiz app, e.g. "General Knowledge Quiz" or "JavaScript Basics Quiz," with 10 questions. Shows a final score and a list of which answers were right or wrong at the end.

Features:

- One question shown at a time with 4 options
- Next button to move to the next question (disable until an option is selected)
- Progress indicator (e.g. "Question 3 of 10")
- Final results screen with score and a review list showing correct vs selected answers
- Restart button to retake the quiz

Focus skills: Managing state across multiple "screens" using JavaScript objects/arrays, conditional rendering by toggling visibility, building a small state machine without a framework.

2.4 — IRCTC-Style Seat Selector (Medium)

2.4 — IRCTC-Style Seat Selector (Medium)

Description: Building on the static Track 1 IRCTC layout, make the seat/class selection interactive — clicking a class (Sleeper, AC 3-Tier, AC 2-Tier) shows the live "fare" and updates a booking summary panel.

Features:

- Clickable class options that highlight when selected
- A live-updating fare summary panel (base fare + selected class price)
- Passenger count selector (+ / − buttons) that recalculates total fare
- "Proceed to Book" button that shows a confirmation message with booking details

Focus skills: Event delegation, dynamic DOM updates based on multiple interacting inputs, calculating and displaying derived values in real time.

2.5 — Expense Tracker with Local Storage (Hard)

Description: A personal finance tracker where users log daily expenses (description, amount, category, date) and see a running total, category-wise breakdown, and a simple bar chart built with plain CSS (no chart library).

Features:

- Add expense form (description, amount, category dropdown, date)
- Expense list showing all entries, newest first, with a delete option per entry
- Summary section: total spent, highest category, this month's total
- A simple CSS-based bar chart showing spending by category
- Data persists across page refreshes using `localStorage`

Focus skills: `localStorage` for persistence, data aggregation logic (grouping by category, summing values), building a chart visualization using only HTML/CSS proportional widths, structuring a larger JavaScript codebase across multiple functions.

Track 3: React + Node.js

Full-stack apps with a React frontend and a Node/Express backend. Data is stored in-memory on the server (arrays/objects) — no database yet, so all data resets when the server restarts.

3.1 — Quote of the Day App (Easy)

Description: A React app that fetches a random quote from a Node/Express backend endpoint every time the user clicks "New Quote." The backend holds an in-memory list

of at least 20 quotes from notable Indian figures (e.g. APJ Abdul Kalam, Swami Vivekananda).

Backend:

- `GET /api/quotes/random` — returns one random quote object `{ text, author }`
- `GET /api/quotes` — returns the full list

Frontend:

- Quote display card with text and author
- "New Quote" button that fetches and updates the display
- Loading state while fetching

Focus skills: First `fetch`/`axios` call from React to Express, `useState` + `useEffect`, basic Express routing.

3.2 — Simple Polling App (Easy)

Description: Users can vote on a single poll question (e.g. "Best programming language for freshers: Java vs Python vs JavaScript") and see live vote counts update after voting.

Backend:

- `GET /api/poll` — returns the question and current vote counts
- `POST /api/poll/vote` — accepts an option and increments its count (in-memory)

Frontend:

- Poll question with clickable option buttons
- After voting, show a simple bar showing percentage per option
- Prevent voting twice using a frontend flag (e.g. disable buttons after voting)

Focus skills: POST requests with a body, updating UI based on server response, percentage calculations.

3.3 — Recipe Finder (Medium)

Description: A React app where users search for recipes by ingredient name (e.g. "paneer," "rice") and the Node backend filters an in-memory recipe list to return matches, similar to searching Indian home-cooking recipes.

Backend:

Backend:

- `GET /api/recipes` — returns all recipes (each with name, ingredients array, steps, cook time)
- `GET /api/recipes/search?ingredient=paneer` — returns recipes containing that ingredient

Frontend:

- Search bar with debounced input (wait until user pauses typing before searching)
- Recipe result cards showing name, ingredient count, and cook time
- Clicking a card opens a detail view with full ingredient list and steps
- "No recipes found" state when search has no matches

Focus skills: Query parameters in Express routes, debouncing in React, conditional rendering of detail vs list views, array filtering logic on the backend.

3.4 — Movie Watchlist Manager (Medium)

Description: Users can search a hardcoded movie catalog (Bollywood and Hollywood titles), add movies to a personal watchlist, mark them watched, and remove them — similar to a simplified IMDb watchlist feature.

Backend:

- `GET /api/movies` — returns the full movie catalog
- `GET /api/watchlist` — returns the current watchlist (in-memory array)
- `POST /api/watchlist` — adds a movie to the watchlist
- `PATCH /api/watchlist/:id` — toggles watched status
- `DELETE /api/watchlist/:id` — removes from watchlist

Frontend:

- Movie catalog browse view with an "Add to Watchlist" button per movie
- Separate Watchlist page/tab showing added movies with watched/unwatched toggle
- Visual distinction between watched and unwatched movies

Focus skills: Full CRUD operations across frontend and backend, React Router (or tab-based view switching) for multiple pages, syncing UI state with server state after each action.

3.5 — Real-Time Code Snippet Sharing Board (Hard)

3.3 — Real-Time Code Snippet Sharing Board (Hard)

Description: A React + Node app similar to a simplified Pastebin — users submit a code snippet with a title and language, and it appears on a shared board for everyone (using polling, not WebSockets, to keep it achievable). Other users can view and copy any snippet.

Backend:

- `GET /api/snippets` — returns all snippets, newest first
- `POST /api/snippets` — adds a new snippet `{ title, language, code, createdAt }`
- `GET /api/snippets/:id` — returns one snippet in full

Frontend:

- Submission form with title, language dropdown, and a code textarea
- Snippet board showing cards with title, language badge, and a preview of the code
- Auto-refresh the board every 10 seconds using `setInterval` so new snippets appear without manual reload
- Detail view with a "Copy to Clipboard" button using the Clipboard API
- Basic syntax-aware styling (monospace font, line numbers using CSS counters)

Focus skills: Polling pattern for near-real-time updates without WebSockets, Clipboard API, larger component structure (list + detail + form), handling concurrent state updates cleanly.

Track 4: React + Node.js + AI

Same full-stack pattern as Track 3, with an AI API call (OpenAI or HuggingFace, your choice) added on the backend. Data is still in-memory — no database required yet, so the focus stays purely on AI integration.

4.1 — Motivational Message Generator (Easy)

Description: A simple app where the user enters a mood or situation (e.g. "feeling demotivated before exams") and the AI generates a short motivational message tailored to it.

Backend:

- `POST /api/motivate` — accepts `{ situation }`, sends a prompt to the AI, returns `{ message }`

Frontend:

- Input field for the situation

- "Get Motivation" button with a loading state while waiting for the AI response
- Display card showing the generated message

Focus skills: First AI API call from a Node backend, prompt writing for a simple single-output task, handling loading and error states in React for an external API call.

4.2 — Text Summarizer (Easy)

Description: Users paste a long paragraph (e.g. a news article or essay) and the AI returns a 2–3 sentence summary.

Backend:

- `POST /api/summarize` — accepts `{ text }`, returns `{ summary }`

Frontend:

- Large textarea for pasting the original text
- Character/word counter
- "Summarize" button
- Summary display, with a "Copy" button

Focus skills: Handling larger text inputs safely, structuring a prompt that constrains output length, basic UI feedback for AI processing time.

4.3 — AI Recipe Suggestion from Ingredients (Medium)

Description: Users type in ingredients they have at home (e.g. "rice, tomato, onion, paneer") and the AI suggests a recipe name, ingredient list, and step-by-step instructions — extending the idea from the Track 3 Recipe Finder, but now generative instead of search-based.

Backend:

- `POST /api/ai-recipe` — accepts `{ ingredients: [] }`, returns structured JSON `{ recipeName, ingredients, steps }`

Frontend:

- Tag-style input where users add ingredients one at a time
- "Suggest Recipe" button
- Result view showing recipe name, ingredient list, and numbered steps
- "Try Another" button to regenerate with the same ingredients

Focus skills: Forcing structured JSON output from the AI (similar to the JobFit AI match-scoring pattern), building a tag input component, designing a prompt that produces consistent step formatting.

4.4 — AI-Powered Interview Question Generator (Medium)

Description: A tool for students preparing for placements — they select a topic (e.g. "React," "DBMS," "Operating Systems") and difficulty level, and the AI generates 5 interview questions with brief expected-answer hints.

Backend:

- `POST /api/interview-questions` — accepts `{ topic, difficulty }`, returns an array of `{ question, hint }` objects

Frontend:

- Topic dropdown and difficulty selector (Beginner / Intermediate / Advanced)
- "Generate Questions" button
- Question list displayed as an accordion — click a question to reveal its hint
- "Generate New Set" button for a fresh batch on the same topic

Focus skills: Generating and rendering arrays of structured AI output, accordion/expand-collapse UI pattern, designing prompts that adjust output based on a difficulty parameter.

4.5 — AI Resume Bullet Point Improver (Hard)

Description: Students paste a weak resume bullet point (e.g. "Worked on a project using React") and the AI rewrites it 3 different ways with stronger, quantifiable, action-oriented phrasing — directly useful for placement season.

Backend:

- `POST /api/improve-bullet` — accepts `{ original }`, returns `{ suggestions: [string, string, string] }`
- Add basic rate-limiting logic (e.g. max 10 requests per session, tracked in-memory) to simulate real-world API cost awareness

Frontend:

- Input field for the original bullet point
- "Improve It" button

- Three suggestion cards, each with a "Copy" button
- A small "Why this works" note generated alongside each suggestion (have the AI explain its reasoning briefly)
- Display of remaining requests left in the session (tied to the backend rate limit)

Focus skills: Designing prompts that return multiple structured variations, building UI for comparing multiple AI outputs side by side, implementing a simple in-memory rate limiter, planning for real-world AI cost behavior even without a database.

Track 5: React + Node.js + MongoDB

Same full-stack pattern, now with real persistence using MongoDB Atlas. Data survives server restarts. Authentication is expected from the Medium level onward.

5.1 — Personal Bookmark Manager (Easy)

Description: A simple app to save, view, and delete useful website links with a title and category (e.g. "Learning," "News," "Tools") — like a personal, simplified version of browser bookmarks.

Backend:

- Mongoose model: `Bookmark { title, url, category, createdAt }`
- `GET /api/bookmarks`, `POST /api/bookmarks`, `DELETE /api/bookmarks/:id`

Frontend:

- Add bookmark form (title, URL, category dropdown)
- Bookmark list grouped by category
- Delete button per bookmark with a confirmation prompt

Focus skills: First Mongoose schema and model, basic CRUD connected to MongoDB Atlas, grouping data in the frontend.

5.2 — Daily Habit Tracker (Easy)

Description: Users define habits they want to track (e.g. "Drink 3L water," "Read 30 mins") and mark each one as done or not done for the current day, with all history saved permanently.

Backend:

- Mongoose model: `Habit { name, createdAt }` and `HabitLog { habitId, date, completed }`

- `GET /api/habits`, `POST /api/habits`
- `POST /api/habits/:id/log` — logs today's completion
- `GET /api/habits/:id/history` — returns completion history

Frontend:

- Add habit form
- Daily checklist view showing today's habits with a toggle
- Simple history view showing a calendar-style grid of past completions (green/grey squares)

Focus skills: Modeling a one-to-many relationship in Mongoose, date handling for "today" vs historical records, rendering a calendar-style grid from array data.

5.3 — College Event Registration System (Medium)

Description: A platform where an admin creates events (tech fests, workshops, seminars) and students register for them with login required, similar to a simplified college event portal.

Backend:

- Mongoose models: `User { name, email, password, role }`, `Event { title, description, date, venue, capacity, registeredUsers }`
- JWT authentication with `admin` and `student` roles
- Admin-only: `POST /api/events`, `PATCH /api/events/:id`
- Student: `GET /api/events`, `POST /api/events/:id/register`
- Capacity check — block registration once an event is full

Frontend:

- Login/signup pages
- Admin dashboard to create and manage events
- Student event browse page with a "Register" button (disabled and labeled "Full" when capacity is reached)
- "My Registrations" page for students showing their registered events

Focus skills: JWT authentication, role-based route protection (matching the JobFit AI admin/candidate pattern), capacity-based business logic, building distinct admin and user-facing views in the same app.

5.4 — Lost and Found Portal for Campus (Medium)

Description: Students can post items they've lost or found on campus, browse other posts, and mark an item as resolved once it's been returned — a genuinely useful tool for any college campus.

Backend:

- Mongoose model: `Post { type ('lost'/'found'), itemName, description, location, postedBy, status, createdAt }`
- JWT authentication required to post
- `GET /api/posts` with optional filters (`?type=lost`, `?location=library`)
- `POST /api/posts`, `PATCH /api/posts/:id/resolve`

Frontend:

- Login/signup
- Post creation form (lost or found toggle, item name, description, location)
- Filterable feed showing all posts, with status badges (Open, Resolved)
- "Mark as Resolved" button visible only to the original poster

Focus skills: Filtering and querying with Mongoose based on multiple optional parameters, ownership-based permission checks (only the poster can resolve their own post), building a real-world community feed UI.

5.5 — Peer-to-Peer Skill Exchange Platform (Hard)

Description: A platform where students list skills they can teach (e.g. "Python basics," "Guitar," "Public speaking") and skills they want to learn. The system matches students who can teach each other, similar to a mini skill-barter marketplace for a college community.

Backend:

- Mongoose models: `User { name, email, password, skillsToTeach[], skillsToLearn[] }`, `MatchRequest { fromUser, toUser, skill, status }`
- JWT authentication
- `GET /api/matches/suggested` — server-side logic to find users whose `skillsToTeach` overlaps with the logged-in user's `skillsToLearn`, and vice versa
- `POST /api/matches/request`, `PATCH /api/matches/:id/accept`, `PATCH /api/matches/:id/decline`
- `GET /api/matches/my-requests` — sent and received requests

Frontend:

Frontend:

- Profile setup — add skills you can teach and skills you want to learn (tag-style inputs)
- "Suggested Matches" page showing potential matches with shared skill highlighted
- Send a connection request with an optional message
- Requests inbox — accept/decline incoming requests, track sent request status
- Once matched, show contact details to both users

Focus skills: Designing a real matching algorithm using array intersection logic in MongoDB queries or backend JavaScript, building a request/accept workflow (similar to a simplified friend-request system), managing multiple relational states (pending, accepted, declined) cleanly across frontend and backend.

Track 6: React + Node.js + MongoDB + AI

The complete stack — exactly like JobFit AI. Persistent data, authentication, and a genuine AI feature that adds real value, not just a gimmick.

6.1 — AI Study Notes Summarizer with History (Easy)

Description: Users paste their class notes or textbook paragraphs, the AI generates a concise summary, and every summary is saved to their personal account so they can revisit it later — turning the Track 4 summarizer into a real persistent tool.

Backend:

- Mongoose models: `User { name, email, password }`, `Summary { userId, originalText, summary, createdAt }`
- JWT authentication
- `POST /api/summarize` — calls the AI, saves the result tied to the logged-in user
- `GET /api/summaries` — returns the logged-in user's summary history

Frontend:

- Login/signup
- Summarizer input page (same as Track 4, now behind login)
- "My Summaries" history page listing all past summaries with date, expandable to view full text

Focus skills: Connecting an AI feature to authenticated, persistent user data — the first

step toward building AI features that remember context across sessions.

6.2 — AI Flashcard Generator for Exam Prep (Easy)

Description: Students paste a topic or paragraph of study material, and the AI generates a set of flashcards (question on front, answer on back). Flashcard sets are saved per user and can be revisited and "studied" later using a flip-card UI.

Backend:

- Mongoose models: `User`, `FlashcardSet { userId, topic, cards: [{ question, answer }], createdAt }`
- JWT authentication
- `POST /api/flashcards/generate` — AI generates 8–10 cards from input text, saves the set
- `GET /api/flashcards` — list of the user's saved sets
- `GET /api/flashcards/:id` — one full set for studying

Frontend:

- Login/signup
- Input page — paste topic/notes, generate flashcards
- "My Sets" page listing saved flashcard sets
- Study mode — one card at a time, click to flip between question and answer, Next/Previous navigation

Focus skills: Forcing the AI to generate a structured array of objects reliably, building a flip-card animation with CSS, organizing AI-generated content into a reusable saved format.

6.3 — AI Career Path Advisor (Medium)

Description: Students fill in their current skills, interests, and academic background. The AI suggests 3 suitable career paths with reasoning, and each suggestion is saved to their profile so they can track how their recommendations evolve as they update their profile over time.

Backend:

- Mongoose models: `User { name, email, password, skills[], interests[], background }`, `CareerSuggestion { userId, suggestions: [{ path, reasoning, suggestedSkillsToLearn }], createdAt }`
- JWT authentication
- `POST /api/career/profile` — update skills/interests/background

- `POST /api/career/advise` — sends profile to AI, returns and saves 3 suggestions
- `GET /api/career/history` — view past suggestion sets to compare over time

Frontend:

- Login/signup
- Profile builder — tag inputs for skills and interests, dropdown for academic background
- "Get Career Advice" button
- Results page — 3 career path cards, each with reasoning and a "skills to learn" list
- History page comparing current and past suggestions side by side

Focus skills: Structuring a multi-field user profile that feeds into an AI prompt, designing prompts that reference multiple structured inputs at once, building a feature that produces evolving value as more user data accumulates.

6.4 — AI Code Review Assistant for Student Projects (Medium)

Description: Students paste a code snippet they've written and get back AI feedback — what's good about it, what could be improved, and a corrected version. All reviewed snippets are saved in a personal portfolio for the student to track their growth over time.

Backend:

- Mongoose models: `User`, `CodeReview { userId, language, originalCode, feedback: { strengths, improvements, correctedCode }, createdAt }`
- JWT authentication
- `POST /api/code-review` — sends code + language to AI, saves structured feedback
- `GET /api/code-review/history` — list of past reviews
- `DELETE /api/code-review/:id` — remove a review from history

Frontend:

- Login/signup
- Code submission page — language dropdown, code textarea
- Feedback display — strengths list, improvements list, corrected code in a styled code block with copy button
- History page — browse past reviews, see code language and date, reopen any past review

Focus skills: Prompting AI for multi-part structured feedback (not just one output field), rendering code blocks cleanly in React, building a feature genuinely useful for a

student's actual coding practice — directly applicable to their own bootcamp work.

6.5 — AI-Powered Mock Interview Platform (Hard)

Description: A complete mock interview simulator. Students select a role (e.g. "Frontend Developer," "Backend Developer," "Data Analyst") and the AI conducts a multi-turn interview — asking a question, accepting the student's typed answer, giving brief feedback, then asking the next question. After 5 questions, the AI generates an overall performance summary saved to the student's profile, building an interview history over time.

Backend:

- Mongoose models: `User`, `InterviewSession { userId, role, questions: [{ question, answer, feedback }], overallSummary, score, createdAt }`
- JWT authentication
- `POST /api/interview/start` — creates a session, AI generates the first question based on role
- `POST /api/interview/:id/answer` — submits an answer, AI gives feedback on that answer AND generates the next question (or ends the session after 5 questions)
- `POST /api/interview/:id/complete` — AI generates an overall summary and score for the full session
- `GET /api/interview/history` — list of all past mock interview sessions with role and score
- `GET /api/interview/:id` — full transcript of one session

Frontend:

- Login/signup
- Role selection screen
- Live interview screen — current question displayed, answer textarea, "Submit Answer" button, shows brief feedback before revealing the next question
- Progress indicator ("Question 3 of 5")
- Final results screen — overall summary, score, and a full transcript with per-question feedback
- "My Interview History" page — list of past sessions by role and score, with ability to revisit any full transcript

Focus skills: Managing a genuinely multi-turn AI conversation with state carried across multiple API calls, designing prompts that maintain context from previous questions and answers within the same session, building a session-based data model in MongoDB

that captures an entire interaction history, and combining everything learned across all six tracks into one cohesive, portfolio-worthy application.

Final Note

Every project in this document, including the Easy ones, results in something you can demo confidently — to a friend, a recruiter, or yourself six months from now when you forget how proud you were the day you finished it. Build one. Finish it completely. Then move to the next.